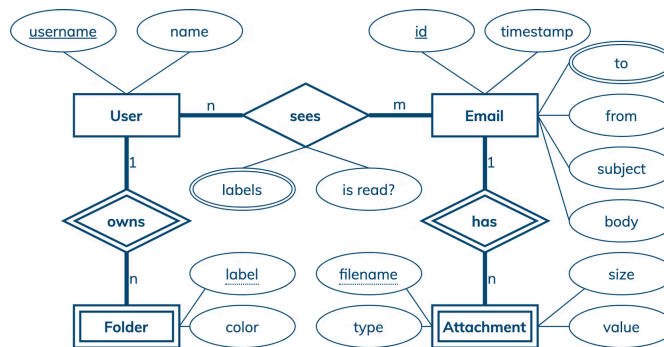


Messaging Data Modeling

Interested in learning more about Cassandra data modeling by example? This example demonstrates how to create a data model for an email system. It covers a conceptual data model, application workflow, logical data model, physical data model, and final CQL schema and query design.

GET STARTED



Entity-Relationship Diagram

Conceptual Data Model

A conceptual data model is designed with the goal of understanding data in a particular domain. In this example, the model is captured using an Entity-Relationship Diagram (ERD) that documents entity types, relationship types, attribute types, and cardinality and key constraints.

The conceptual data model for messaging data features users, folders, emails and email attachments. A user is identified by a unique username and may have other attributes like name. A folder has a label and color, and is uniquely identified by a combination of a label and username. An email has a unique id, timestamp, one or more recipients, one sender, subject and body. While a user can own many folders, each folder can only belong to one user. Similarly, an email can have many attachments, but an attachment always belongs to exactly one email. Finally, a user can have multiple emails and each email can be seen by multiple users. Since an email can have many labels, it can appear in many folders.

Next: Application Workflow

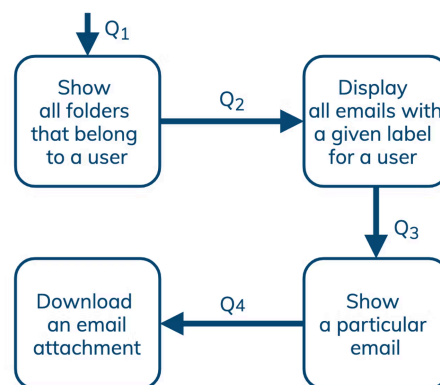
Application Workflow

An application workflow is designed with the goal of understanding data access patterns for a data-driven application. Its visual representation consists of application tasks, dependencies among tasks, and data access patterns. Ideally, each data access pattern

should specify what attributes to search for, search on, order by, or do aggregation on.

First, the application workflow has an entry-point task that shows all folders that belong to a particular user. This task requires querying a database to find information about folder labels, colors and unread email quantities for a given user, which is documented as Q1 on the diagram. Second, an application can proceed to display all emails with a given label based on a user folder selection, which requires data access pattern Q2. The resulting list of emails should be sorted using email timestamps, showing the most recent emails at the top. Third, the next task can show all information about an individual email selected by a user, which requires data access pattern Q3. Finally, the task of downloading an individual email attachment is based on data access pattern Q4. All in all, there are four data access patterns for a database to support.

Next: Logical Data Model



Data access patterns

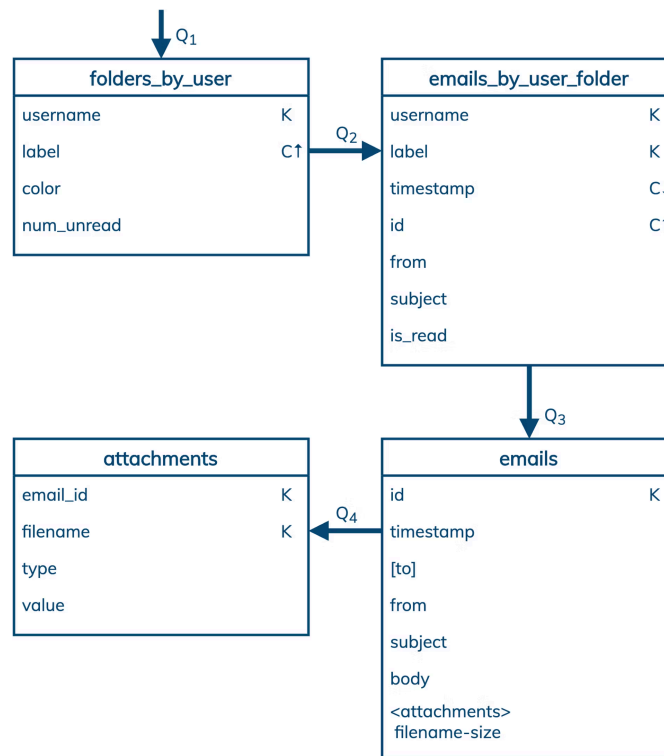
Q1: Find information about folder labels, colors and unread email quantities for a given user

Q2: Find ids, subjects, senders, read/unread statuses and timestamps of all emails with a known label for a given user; order by timestamp (desc)

Q3: Find all available information about a particular email

Q4: Find an attachment file for a particular email

Application Workflow Diagram



Chebotko Diagram

Logical Data Model

A logical data model results from a conceptual data model by organizing data into Cassandra-specific data structures based on data access patterns identified by an application workflow. Logical data models can be conveniently captured and visualized using Chebotko Diagrams that can feature tables, materialized views, indexes and so forth.

The logical data model for messaging data is represented by the shown Chebotko Diagram. There are four tables, namely **folders_by_user**, **emails_by_user_folder**, **emails** and **attachments**, that are designed to specifically support data access patterns Q1, Q2, Q3 and Q4, respectively. Table **folders_by_user** is designed to have a separate partition for each user, and each partition can contain multiple rows capturing information about individual folders. Therefore, Q1 can be satisfied by retrieving all rows from one partition. Table **emails_by_user_folder** has a composite partition key, consisting of columns **username** and **label**, and a composite clustering key, consisting of columns **timestamp** and **id**. It is designed to store all emails that belong to the same folder in one partition, where each individual email maps to a row. Similarly to Q1, Q2 can be satisfied by accessing only one partition. Finally, tables **emails** and **attachments** are single-row partition tables that are designed to store one email or one attachment per partition, respectively. Access patterns Q3 and Q4 require retrieving one row from one partition. While this design is straightforward, notice how each email or attachment is intended to be stored only one time, even though they can be accessed by many users via many folders.

Next: Physical Data Model

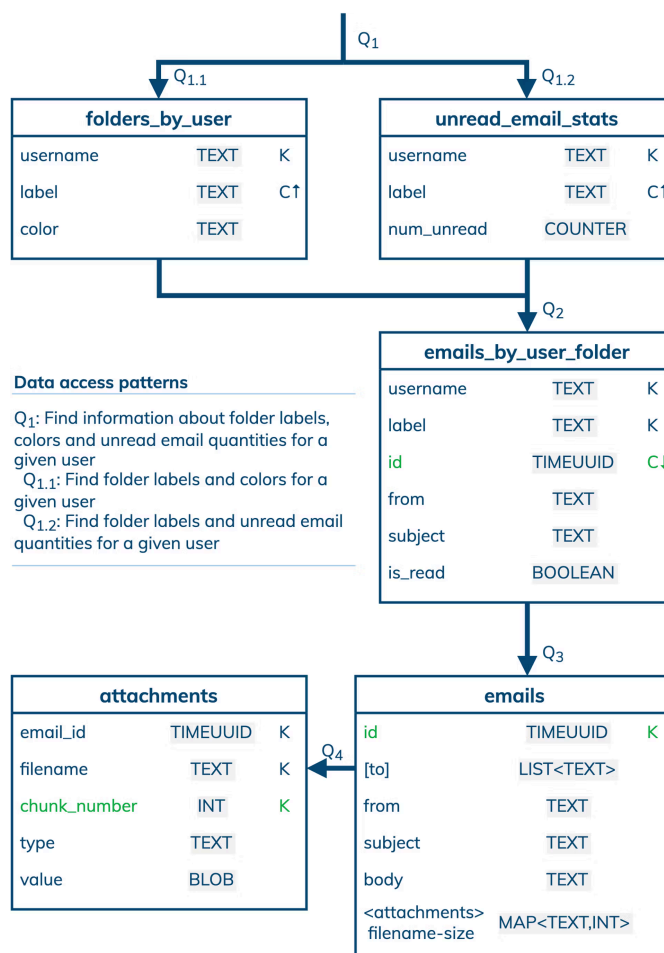
Physical Data Model

A physical data model is directly derived from a logical data model by analyzing and optimizing for performance. The most common type of

analysis is identifying potentially large partitions. Some common optimization techniques include splitting and merging partitions, data indexing, data aggregation and concurrent data access optimizations.

The physical data model for messaging data is visualized using the Chebotko Diagram. This time, all table columns have associated data types. In addition, every table has some column-related changes, and there is even one new table. Table `folders_by_user` no longer has column `num_unread` as it is now part of new table `unread_email_stats`. The new table is necessary to be able to use the COUNTER data type: any table with one or more counter columns cannot have non-counter columns other than primary key columns. Tables `emails_by_user_folder` and `emails` no longer have separate columns to store email timestamps because, in both cases, timestamps can be easily extracted from column `id` of type `TIMEUUID`. Furthermore, in the case of table `emails_by_user_folder`, the clustering order is going to be based on timestamps. It is worth mentioning that partitions in table `emails_by_user_folder` can grow over time to become very large. However, instead of introducing a new column into the partition key, overflow labels can be used. For example, if a folder with the inbox label becomes too big, the system can automatically start using a new label like `inbox-overflow-1` to store more emails, which should be transparent to the user. Finally, column `chunk_number` is introduced into the partition key of table `attachments` to be able to divide large attachments into smaller chunks and store them separately. For example, assuming the chunk size limit of 1000KB, a 530KB file can be stored as one chunk and a 2416KB file has to be stored using three chunks. This optimization helps to store and retrieve large attachments faster since different nodes in a cluster may be able to handle different chunks in parallel. Our final blueprint is ready to be instantiated in Cassandra.

Skill Building



Chebotko Diagram

Skill Building

Want to get some hands-on experience? Give our interactive lab a try!
You can do it all from your browser, it only takes a few minutes and you don't have to install anything.

Start Hands-on Lab

(A [GitHub](#) account may be required to run this lab in Gitpod.
Supported browsers: Firefox, Chrome/Chromium, Edge, Brave.)

More Resources

Review the Cassandra data modeling process with these videos

[会社情報](#)[会社概要](#)[リーダーシップ](#)[エグゼクティブ マネジメント チーム](#)[お問い合わせ](#)[リソース](#)[ドキュメント](#)[ブログ](#)[イベント](#)[Bodiのご紹介](#)

パートナー

採用

ニュース

ブランドリソース

コンタクト・サポート

セキュリティ

ベクトルデータベースとは？

Retrieval-Augmented Generation (RAG)とは？

クラウド・パートナー

Amazon Web Services

Google Cloud

Microsoft Azure

NVIDIA

AI++を購読

AIを使ってアプリケーションを構築する開発者のためのニュースレター

Work Email

➤

© 2025 DataStax

[個人情報保護方針](#) | [利用規約](#) | [商標について](#) | [法務情報](#) | [プライバシー設定の管理](#)

Japan